

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · Sistemas Inteligentes

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Leonardo Ricci Santos Oliveira

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 06/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	5
4.3	Atividade 3 — Desacoplando o login — microserviço de autenticação	7
4.4	Atividade 4 — Controle de acesso por papel — RBAC	11
4.5	Atividade 5 — Logs e auditoria	15
4.6	Atividade 6 — Upload e perfil de usuário	17
5	Atividades extras (opcional)	19
5.1	Atividade E1 — Documentação Swagger/OpenAPI	19
5.2	Atividade E2 — CI/CD com GitHub Actions	22
5.3	Atividade E3 — Observabilidade — health checks e métricas	24
6	Melhorias de UX/UI	26
6.1	Página inicial pública	26
6.2	Identidade visual e telas de login e cadastro	26
6.3	Catálogo: busca, paginação e filmes sem pôster	27
6.4	Detalhes do filme	28
6.5	Menu do usuário	28
7	Considerações finais	28
8	Declaração de autoria	30

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055) trata de como uma aplicação deixa de ser um programa que roda na máquina de quem o escreveu e passa a ser um sistema publicado, composto por várias partes independentes que conversam pela rede. Os temas vão do empacotamento em containers (Docker) à separação em serviços, controle de acesso, registro de auditoria, armazenamento de objetos e automação de deploy. A ideia é que cada conceito apareça aplicado num sistema que já existe, e não como exercício isolado.

O fio condutor do bimestre foi um único projeto: um catálogo de filmes do Tom Hanks. Depois de uma atividade de nivelamento em sala (uma agenda telefônica simples em Flask), o catálogo nasceu como uma API FastAPI com frontend Angular, que busca os filmes ao vivo na API pública da TMDb e permite que cada usuário favorite e comente filmes sem enxergar os dados dos outros. A cada atividade o mesmo sistema cresceu: a autenticação virou um microsserviço próprio, os papéis de usuário passaram a decidir permissões no servidor, as ações sensíveis passaram a ser auditadas num serviço de logs com Redis e os usuários ganharam um perfil com foto guardada em object storage. Por fim, nas atividades extras (opcionais e sem prazo), entraram documentação OpenAPI, um pipeline de CI/CD e observabilidade.

Este relatório reúne essas entregas. A Metodologia explica como o trabalho foi feito e de onde saíram as datas. O Quadro de entregas resume, numa tabela, data planejada, data realizada e situação de cada atividade. Em seguida vem uma ficha por atividade, com o que foi construído, as evidências (link do commit e prints) e as dificuldades encontradas. Depois das fichas, uma seção compara as telas do sistema antes e depois das melhorias de interface. O relatório termina com as considerações finais, que trazem um diagrama da arquitetura atual, e a declaração de autoria.

2 Metodologia

A atividade 1 foi feita em sala e versionada depois no repositório `study_infraestructure`. As atividades 2 a 6 e as extras foram desenvolvidas no mesmo repositório público do GitHub (`leonardoricci-tsi/api_filme_`), como evolução contínua de um único projeto. O ambiente foi o de desenvolvimento local com Docker Compose (catálogo, auth-service, log-service, Redis e Garage na mesma rede interna) e, a partir da atividade 3, também o servidor da disciplina, gerenciado pelo Portainer (`docker-compose.portainer.yml`). O banco relacional é uma instância MySQL remota fornecida pela disciplina, compartilhada pelo catálogo e pelo auth-service. Cada etapa foi feita em commits pequenos, acompanhados de testes automatizados com `pytest`.

As datas e horas de entrega foram extraídas do histórico de commits com `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad %s" <hash>` e conferidas na página do commit no GitHub e no arquivo `.patch` que o próprio GitHub serve para cada commit, que mostra a data com fuso horário. Todos os commits usados já estão no fuso de Brasília (-03:00), então não houve conversão; em todos eles a data de autoria coincide com a data do commit. O critério para escolher o commit de cada atividade foi o momento em que a funcionalidade pedida passou a funcionar completa. Commits anteriores (partes da implementação) e posteriores (correções, documentação e evidências) são citados na ficha de cada atividade. A situação compara essa data com a data planejada pelo professor, considerando o prazo até as 23:59 do dia.

As evidências são de dois tipos. O print da entrega mostra a página do commit no GitHub (repositório e hash) junto com o cabeçalho do `.patch` do mesmo commit (hash completo, data e hora). Esse cabeçalho foi incluído porque a página do commit só mostra o dia, e a hora aparece apenas ao passar o mouse sobre a

data. O print do resultado mostra o sistema funcionando. Nas atividades 3 a 6 e nas extras, são as capturas feitas na época de cada entrega, já versionadas na pasta docs/evidencias/ do repositório e copiadas para a pasta prints/ deste relatório. Os prints de resultado das atividades 1 e 2 e as comparações da seção de UX/UI foram feitos durante a elaboração deste relatório, executando o código de cada versão.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	Em sala (versionada em 14/08/2026 19:59)	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	16/08/2026 19:09	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 17:15	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	02/09/2026 17:25	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	25/09/2026 13:49	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	29/09/2026 20:37	entregue
E1	Documentação Swagger/ OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	25/09/2026 15:04	entregue
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	30/09/2026 15:34	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	30/09/2026 18:29	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
------------------	---

Data planejada	07/08/2026
Data realizada	Em sala (versionada em 14/08/2026 19:59)
Situação	entregue
Evidência	Realizada em sala — print do sistema rodando localmente; código versionado depois no GitHub (commit 527ebdb)
Link	https://github.com/leonardoricci-tsi/study_infraestructure/commit/527ebdb

O que foi feito. Uma agenda monolítica em Flask, num único arquivo (`app.py`). A rota `/` mostra um formulário (nome, telefone e valor) e a lista de cadastros, renderizada com um template Jinja embutido no código (`render_template_string`). Os registros são gravados num arquivo `dados.json`, sem banco de dados. Há também duas rotas JSON, `POST /salvar` e `GET /listar`, que fazem o mesmo trabalho para um cliente que não seja o navegador. A rota `/` aceita tanto formulário HTML quanto JSON, e a listagem acrescenta um campo `enderecos` aos registros antigos que ainda não o tinham.

A atividade foi feita em sala. O código foi enviado ao GitHub depois, em 14/08/2026 às 19:59 (commit 527ebdb, único do repositório), numa época em que versionar a atividade ainda não era obrigatório. A evidência da entrega é o sistema rodando.

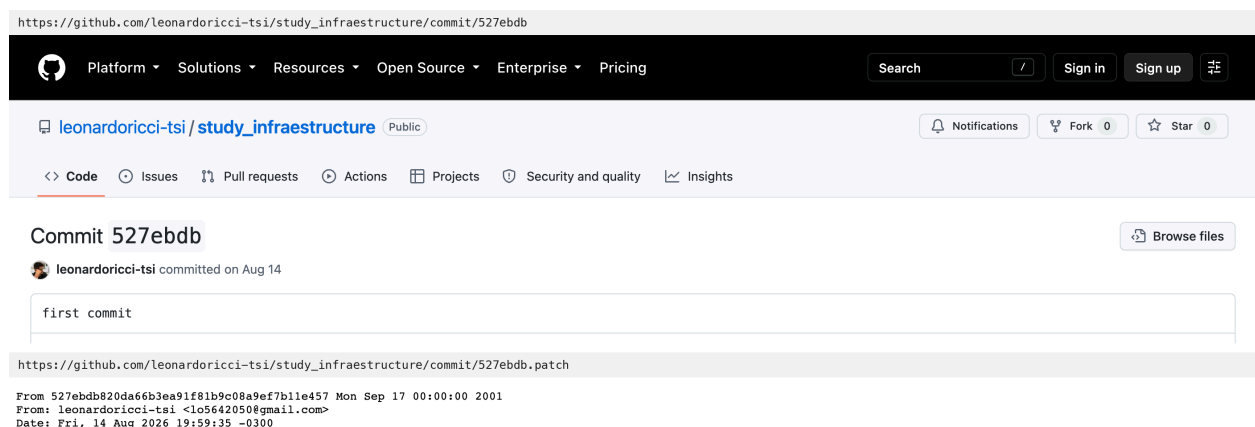
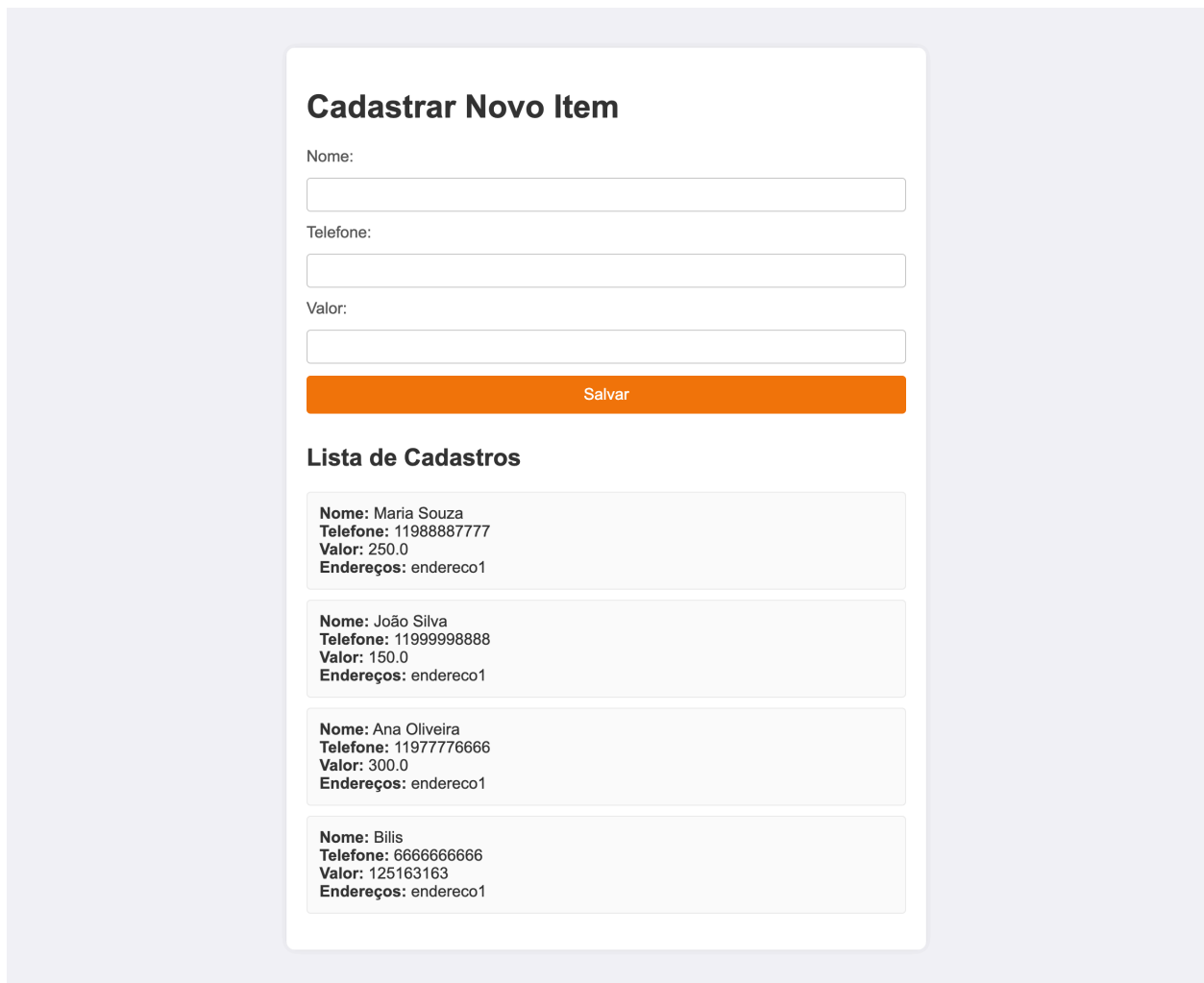


Figura 1: Atividade 1 — commit 527ebdb no GitHub (repositório, hash, data e hora)



Cadastrar Novo Item

Nome:

Telefone:

Valor:

Salvar

Lista de Cadastros

- Nome: Maria Souza
Telefone: 11988887777
Valor: 250.0
Endereços: endereco1
- Nome: João Silva
Telefone: 11999998888
Valor: 150.0
Endereços: endereco1
- Nome: Ana Oliveira
Telefone: 11977776666
Valor: 300.0
Endereços: endereco1
- Nome: Bilis
Telefone: 66666666666
Valor: 125163163
Endereços: endereco1

Figura 2: Atividade 1 — agenda rodando localmente (formulário e lista de cadastros)

Dificuldades e como foram resolvidas. A maior dificuldade foi juntar back-end, front-end e banco de dados ao mesmo tempo num código monolítico e fazer as três partes funcionarem de forma coordenada. Resolvi isso concentrando tudo em `app.py` e separando as responsabilidades em funções pequenas. `carregar_dados()` e `salvar_dados()` são a única porta de entrada para o arquivo `dados.json`, que faz o papel de banco. O HTML é um template Jinja dentro do próprio código, renderizado com `render_template_string`. A mesma rota `/` atende tanto o formulário do navegador quanto requisições JSON (`request.is_json`), e as rotas `/salvar` e `/listar` reaproveitam as mesmas funções de leitura e gravação. Assim, todas as partes passam pelo mesmo caminho até os dados.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	16/08/2026 19:09
Situação	entregue
Evidência	GitHub — commit + README mencionando github.com/siriani (desde b1bdb48) + print do catálogo
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/4999e4e

O que foi feito. Uma API em FastAPI com SQLAlchemy e Alembic e um frontend Angular (componentes standalone), empacotados num Dockerfile multi-stage: o build do Angular é servido como arquivo estático pelo próprio FastAPI. Os filmes do Tom Hanks vêm sempre ao vivo da TMDb e nunca são gravados no banco. O banco guarda só o que é do usuário: conta, favoritos e comentários. O cadastro e o login usam senha com hash bcrypt e sessão por JWT. O isolamento entre usuários é feito no backend: toda consulta de favoritos e comentários é filtrada pelo `usuario_id` extraído do token, e tentar mexer num registro de outra pessoa é recusado (`app/routers/_ownership.py`). O crédito ao professor (github.com/siriani) foi adicionado ao README no commit `b1bdb48`, às 19:14 do mesmo dia.

Em vez de MariaDB, o projeto usa a instância MySQL remota que a disciplina fornece a cada aluno (driver `pymysql`, `DATABASE_URL=mysql+pymysql://...`). Os dois bancos são compatíveis no SQL usado pelo SQLAlchemy e pelas migrations, e o `docker-compose.yml` não sobe nenhum container de banco. O /health público de produção mostra a checagem do MySQL (última figura desta ficha).

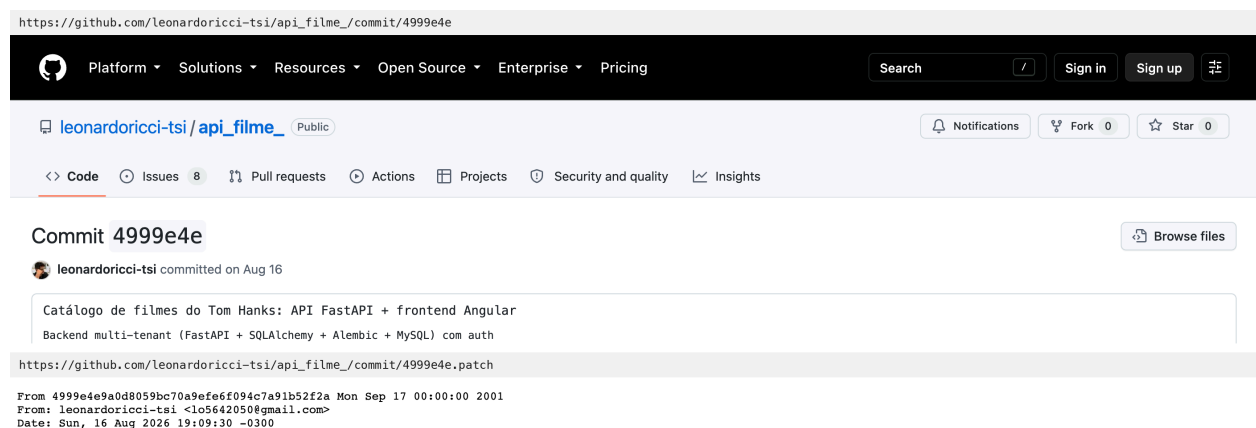


Figura 3: Atividade 2 — commit 4999e4e no GitHub (repositório, hash, data e hora)

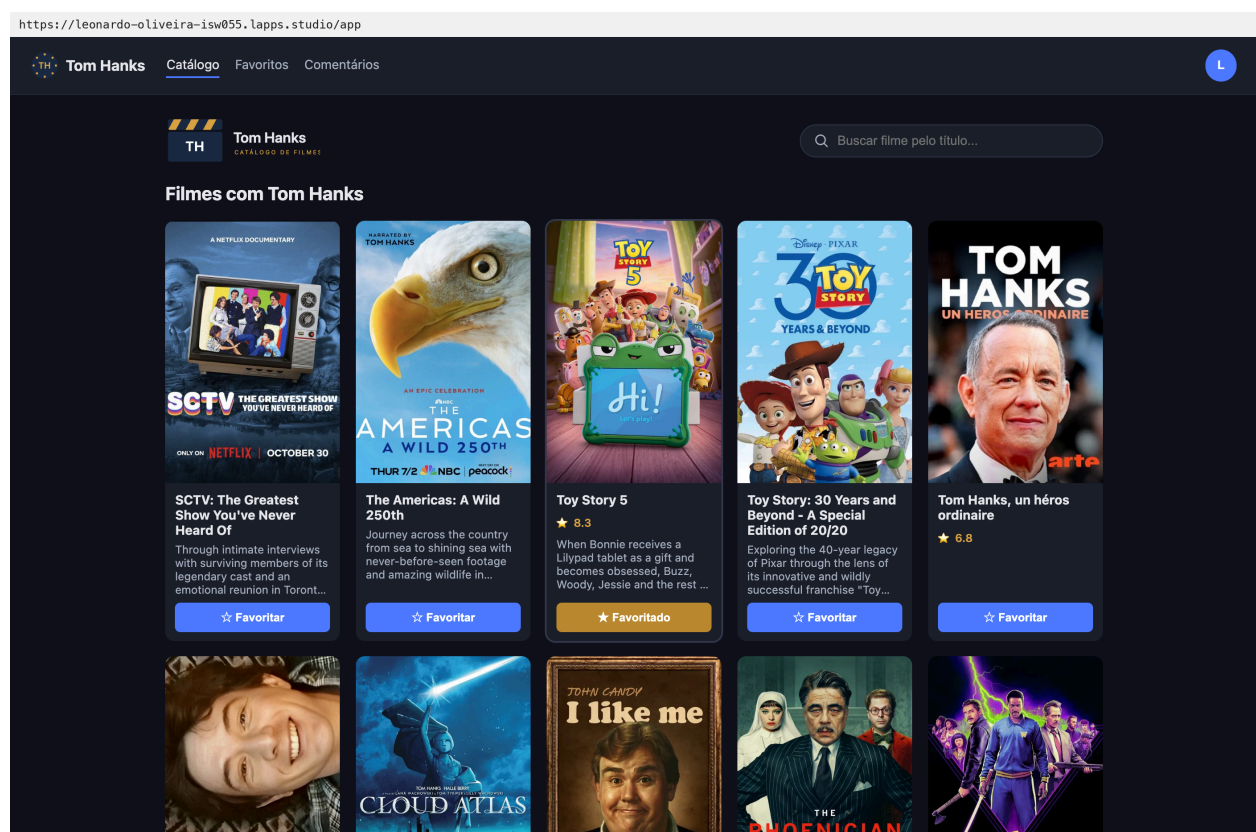


Figura 4: Atividade 2 — resultado: catálogo de filmes do Tom Hanks com usuário logado (filme favoritado em destaque)

```
https://leonardo-oliveira-isw055.lapps.studio/health
Pretty print ☐
{"status":"ok","checagens":{"mysql":{"status":"ok","critica":true,"ms":300},"auth-service":{"status":"ok","critica":false,"ms":28},"log-service":{"status":"ok","critica":false,"ms":21},"garage":{"status":"ok","critica":false,"ms":6}}}
```

Figura 5: Atividade 2 — GET /health em produção: checagem do MySQL remoto (crítica) com status ok

```
https://github.com/leonardoricci-tsi/api_filme_ (README.md)
```

Catálogo de Filmes — Tom Hanks (multi-tenant)

CI passing

Atividade da disciplina, proposta pelo professor [@siriani](#).

API FastAPI que lista filmes do Tom Hanks (dados sempre ao vivo da TMDb, nunca persistidos) e permite que cada usuário cadastrado no app favorite e comente filmes, com isolamento total de dados entre usuários proposto pelo professor [@siriani](#).

Atividade 3 — serviços desacoplados: a autenticação (login, cadastro, papéis de usuário e recuperação de senha) foi extraída do catálogo para um microserviço próprio, o `auth-service/`, que só é alcançável pela rede interna do Docker. O catálogo continua sendo o único ponto de entrada público. Veja a seção [Arquitetura](#) e as [evidências](#) mais abaixo.

Atividade 6 — armazenamento de objetos: o catálogo virou uma rede social simples: cada usuário tem uma página de perfil com foto, bio e os filmes favoritos. A foto **não vai pro banco** — vai pra um object storage (Garage, compatível com S3), e o MySQL guarda só a chave do objeto. Veja a seção [Perfil e fotos](#).

Atividade extra — CI/CD com GitHub Actions: deploy sem clicar em nada. A cada `git push` o GitHub Actions

Figura 6: README do repositório com a menção ao professor (github.com/siriani), presente desde o commit b1bdb48 (16/08/2026 19:14)

Dificuldades e como foram resolvidas. A atividade juntou integração com a TMDb, autenticação com JWT, migrations e deploy. O mais difícil foram as migrations com Alembic: eu não conhecia a biblioteca e precisei estudar o essencial para criar e aplicar as migrations. Configurei o `alembic/env.py` para ler a `DATABASE_URL` das mesmas configurações da aplicação (`get_settings()`) e para usar os modelos do SQLAlchemy como referência (`target_metadata = Base.metadata`). Assim, o esquema do banco nasce dos modelos, sem SQL escrito à mão. A primeira migration (`6c7cac7f3161_initial_schema.py`) cria `usuarios`, `favoritos` e `comentarios`, com a restrição `UNIQUE(usuario_id, tmdb_movie_id)` em `favoritos`. Depois vieram duas migrations incrementais (`9286b70fe465` e `ac04bc85ee9c`, que adicionam `poster_path` e `titulo` em `comentarios`), o que mostra o fluxo de evoluir o banco sem recriá-lo. Para o deploy, as migrations não rodam sozinhas na subida do container: são aplicadas com `alembic upgrade head`, localmente ou com `docker run --rm --env-file .env api-filmes alembic upgrade head`, como documentado no README. A partir da atividade 3, esse passo virou o serviço `migrate` do `docker-compose.yml`.

4.3 Atividade 3 — Desacoplando o login — microserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026

Data realizada	27/08/2026 17:15
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando (e do fluxo de recuperação de senha)
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/499cb42

O que foi feito. A autenticação saiu do catálogo e virou o auth-service/, também em FastAPI. Ele cuida de cadastro, login, papel do usuário e recuperação de senha. No docker-compose.yml ele não tem ports:, só expõe: "8001", então só é alcançável pela rede interna catalog-net, pelo nome http://auth-service:8001. O catálogo continua sendo o único ponto de entrada público: toda rota /auth/* chega nele e é repassada internamente (app/services/auth_client.py). Nas demais rotas o catálogo não consulta o auth-service: o JWT é assinado por ele e verificado localmente pelo catálogo, com o mesmo JWT_SECRET, o que evita uma ida pela rede a cada requisição.

A recuperação de senha gera um token aleatório (secrets.token_urlsafe(32)) com validade de 30 minutos e uso único, gravado na tabela reset_tokens. O link é enviado por e-mail via SMTP (Mailtrap em desenvolvimento). O pedido sempre devolve a mesma mensagem, exista ou não o e-mail, para não revelar quais e-mails têm conta. A entrega foi feita em etapas: bd536aa (docker compose), 72a259b (scaffold do serviço), db99632 (catálogo vira proxy), 2064573 e 258405e (token de redefinição), f21f1c2 (rotas no catálogo), f307bb0 (envio real de e-mail) e 499cb42 (telas do Angular), commit em que o fluxo passou a funcionar de ponta a ponta. README e evidências vieram em 4ecfbcf, às 17:26 do mesmo dia.

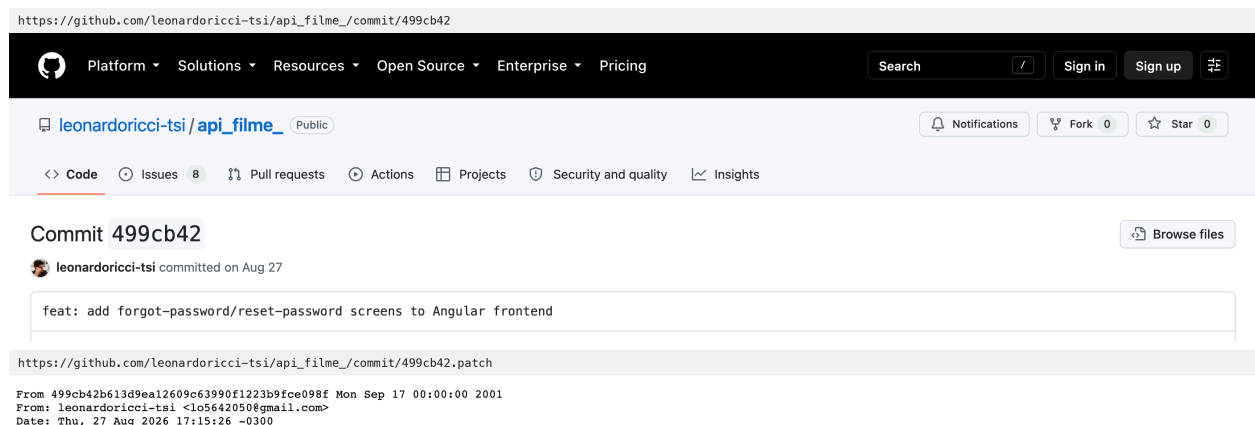


Figura 7: Atividade 3 — commit 499cb42 no GitHub (repositório, hash, data e hora)

```
https://github.com/leonardoricci-tsi/api_filme_/blob/499cb42/docker-compose.yml

leonardoricci-tsi / api_filme_

<> Code Issues 8 Pull requests Actions Projects Security and quality Insights

api_filme_ / docker-compose.yml

leonardoricci-tsi feat: cria microsserviço de autenticação (scaffold) 72a259b · 2 months ago History

Code Blame 53 lines (49 loc) · 1.1 KB Raw

1 networks:
2   catalog-net:
3     driver: bridge
4
5 services:
6   api:
7     build: .
8     ports:
9       - "${PORT:-8000}:8000"
10    env_file:
11      - .env
12    environment:
13      AUTH_SERVICE_URL: http://auth-service:8001
14    networks:
15      - catalog-net
16    depends_on:
17      - auth-service
18
19  migrate:
20    build: .
21    env_file:
22      - .env
23    entrypoint: ["alembic"]
24    command: ["upgrade", "head"]
25    networks:
26      - catalog-net
27    profiles:
28      - tools
29
30  # Serviço de autenticação: login, cadastro, papéis (role) e (a partir da
31  # fase 3) esqueci-minha-senha. De propósito SEM `ports:` — não é
32  # alcançável do host, só de dentro da rede `catalog-net`, pelo nome do
33  # serviço (http://auth-service:8001). O catálogo (`api`) é o único
34  # container com porta publicada pro host.
35  auth-service:
36    build: ./auth-service
37    env_file:
38      - .env
39    expose:
40      - "8001"
41    networks:
42      - catalog-net
43
44  migrate-auth:
45    build: ./auth-service
46    env_file:
47      - .env
48    entrypoint: ["alembic"]
49    command: ["upgrade", "head"]
50    networks:
51      - catalog-net
52    profiles:
53      - tools
```

Figura 8: Atividade 3 — docker-compose.yml no commit 499cb42: o catálogo (api) publica a porta; o auth-service só tem expose: "8001", na rede catalog-net

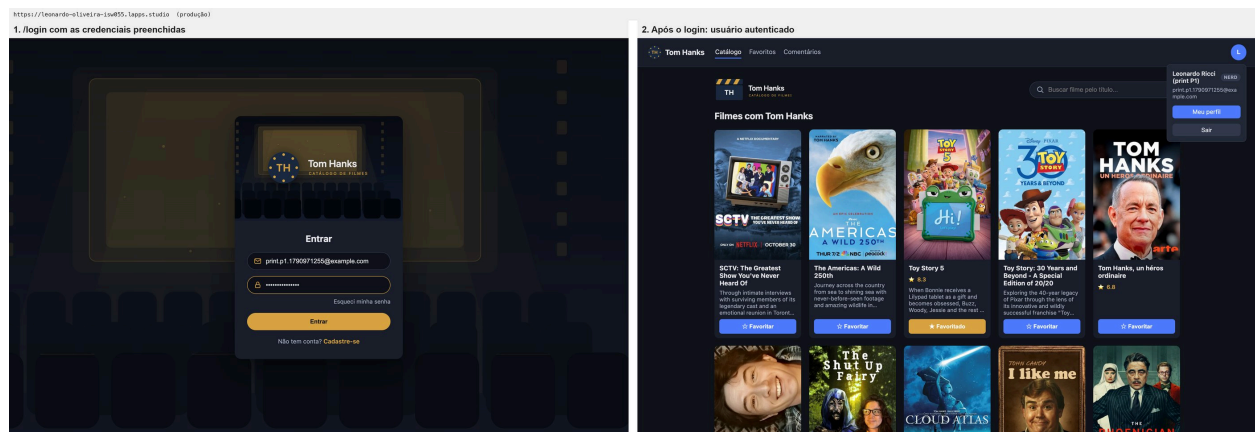


Figura 9: Atividade 3 — login funcionando em produção: credenciais enviadas ao catálogo, repassadas ao auth-service, e usuário autenticado

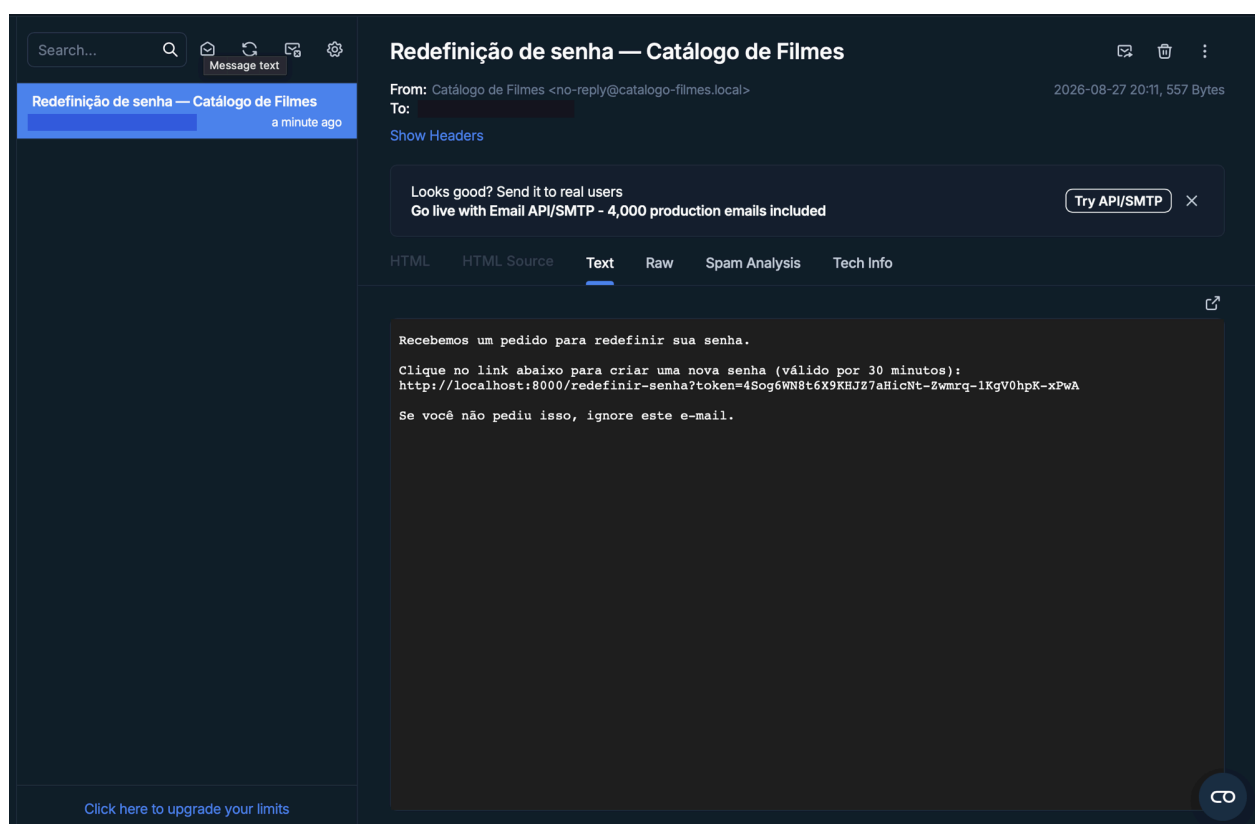


Figura 10: Atividade 3 — e-mail de redefinição de senha recebido no Mailtrap

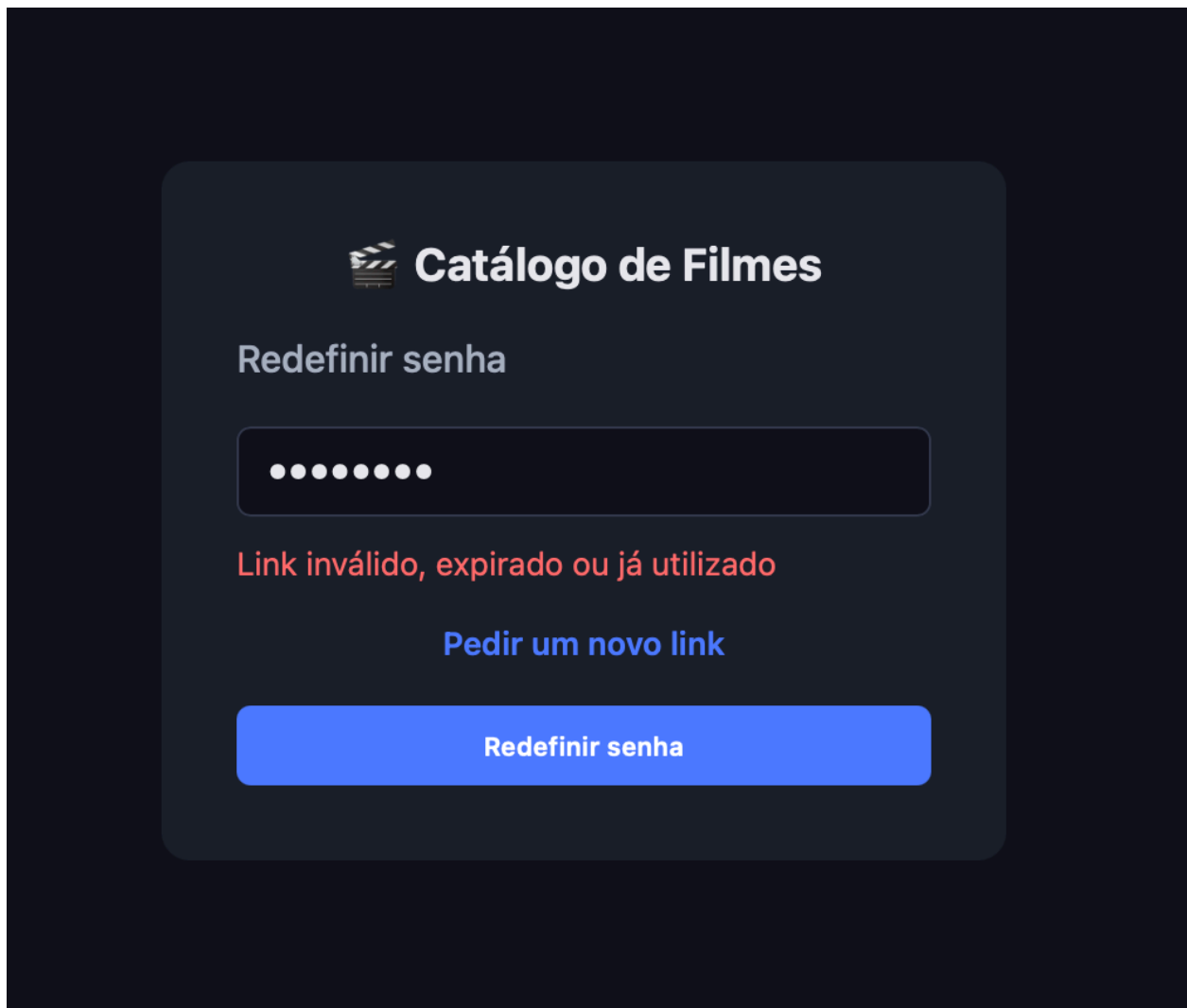


Figura 11: Atividade 3 — link reutilizado é recusado (token de uso único)

Dificuldades e como foram resolvidas. O login, que fazia parte do monolito, foi separado num container Docker próprio, e implementei o fluxo de “esqueci minha senha” com envio de e-mail. A parte mais complicada foi integrar o Brevo ao código via SMTP, o que exigiu pesquisar como fazer essa integração. A solução foi não amarrar o código a nenhum provedor: `auth-service/app/services/mailer.py` usa a biblioteca padrão do Python (`smtplib`), abre a conexão, ativa a criptografia com `starttls()`, faz login e envia a mensagem montada com `EmailMessage`. Host, porta, usuário, senha e remetente vêm de variáveis de ambiente (`SMTP_*` em `auth-service/app/config.py`). Em desenvolvimento elas apontam para o Mailtrap, e em produção as mesmas variáveis, cadastradas na stack do Portainer (`docker-compose.portainer.yml`), apontam para o Brevo, sem mudar nenhuma linha de código. Uma falha de envio é registrada no log do container e não derruba a requisição, e a resposta ao usuário é sempre a mesma mensagem genérica. Outro ponto foi o banco: como a disciplina fornece um único banco por aluno, o `auth-service` reaproveita a mesma instância MySQL com um histórico de migrations próprio, numa tabela de versão separada (`VERSION_TABLE` em `auth-service/alembic/env.py`).

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo <code>role</code> passa a decidir permissões reais no backend (403 para usuário comum).
-----------	---

Data planejada	04/09/2026
Data realizada	02/09/2026 17:25
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/bd02f64

O que foi feito. Quatro papéis em escada, cada um incluindo as permissões do anterior: cinefilo (padrão no cadastro; só vê o catálogo), nerd (comenta e favorita), stalker_do_tomhanks (joga o quiz do pôster pixelado) e admin. Só o admin pode apagar comentários e favoritos de qualquer usuário e listar, promover e rebaixar usuários. A decisão é sempre do servidor. `require_papel_minimo` compara o nível do papel (`NIVEL_PAPEL`) e `require_admin` exige o papel exato. Os dois leem o `claim role` do JWT localmente (Padrão B), e quem não tem o papel recebe 403 mesmo chamando a API direto por curl ou Postman. A gestão de usuários fica no `auth-service`, que é o dono da tabela `usuarios` e faz a própria checagem de admin. O quiz confere a resposta só no servidor, usando um `round_id` assinado que o cliente não consegue forjar.

Sequência no histórico, toda em 02/09: 349be5b (papéis), a472aa2 (moderação e gestão de papéis com 403 no backend, mais os testes em `tests/test_admin.py`), 2652d1b (quiz), 8f4ee59 (README e prints do 403 e do 200), a50ae97 e 047ca7f (papéis e quiz no frontend), d247b73 (correção) e bd02f64 (painel de admin), que fecha a atividade.

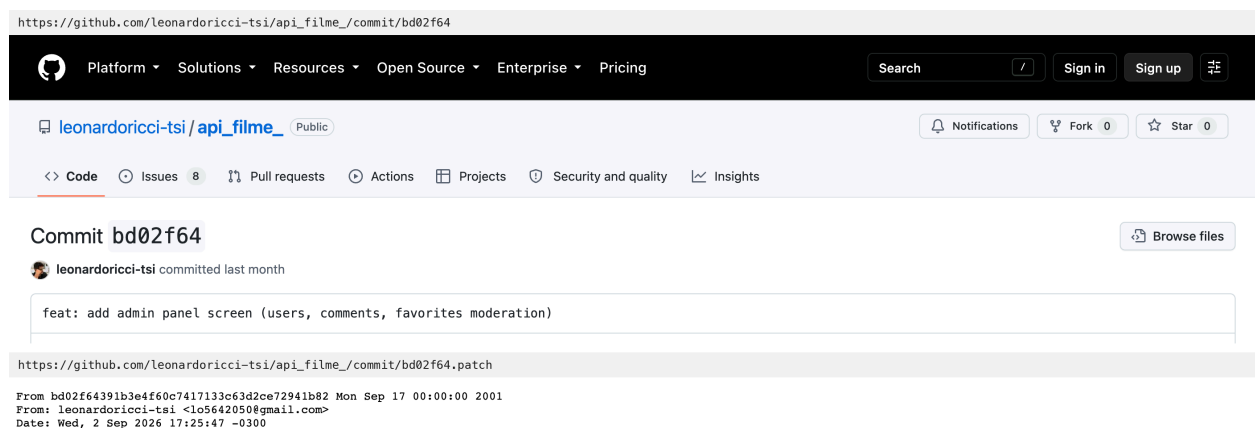


Figura 12: Atividade 4 — commit bd02f64 no GitHub (repositório, hash, data e hora)

PATCH /auth/admin/users/{usuario_id}/role Alterar Papel Usuario

Parameters

Cancel

Reset

Name	Description
usuario_id • required integer (path)	21

Request body required

application/json

Edit Value | Schema

```
{  
  "role": "nerd"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \  
  'http://127.0.0.1:8000/auth/admin/users/21/role' \  
  -H 'accept: application/json' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyMSIsInJvdGUiOiJuZXJkIiwibm9tZSI6IiVzdWYyaW8g'  
  -H 'Content-Type: application/json' \  
  -d '{  
    "role": "nerd"  
  }'
```

Request URL

http://127.0.0.1:8000/auth/admin/users/21/role

Server response

Code	Details
403 <i>Undocumented</i>	Error: Forbidden Response body <pre>{ "detail": "Acesso restrito a admins" }</pre> Download Response headers <pre>content-length: 37 content-type: application/json date: Wed, 02 Sep 2026 18:45:27 GMT server: uvicorn</pre>

Figura 13: Atividade 4 — usuário comum recebe 403 ao tentar uma ação de admin

PATCH

/auth/admin/users/{usuario_id}/role Alterar Papel Usuario

Parameters

Cancel

Reset

Name	Description
usuario_id * required integer <i>(path)</i>	21

Request body required

application/json

Edit Value | Schema

{
 "role": "nerd"
}

Execute

Clear

Responses

Curl

curl -X 'PATCH' \
 'http://127.0.0.1:8000/auth/admin/users/21/role' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyMiIsInVjbGU0IjohZG1pbiIsIm5vbmUiOiJBZG1pbiBS'
-H 'Content-Type: application/json' \
-d '{
 "role": "nerd"
}'

Request URL

http://127.0.0.1:8000/auth/admin/users/21/role

Server response

Code	Details
200	Response body { "id": 21, "name": "Usuario Comum RBAC", "email": "cinefilo.rbac.1788305527@example.com", "role": "nerd", "criado_em": "2026-09-01T23:32:11" } Download Response headers content-length: 132 content-type: application/json date: Wed, 02 Sep 2026 18:43:22 GMT server: uvicorn

Figura 14: Atividade 4 – a mesma ação executada com sucesso pelo admin (200)

Dificuldades e como foram resolvidas. O desafio principal foi definir as regras de negócio: pensar nas particularidades de cada papel, no que cada um pode fazer e em todos os cenários que poderiam dar certo ou errado. Resolvi primeiro organizando os papéis numa escada (`NIVEL_PAPEL = {"cinefilo": 1, "nerd": 2, "stalker_do_tomhanks": 3, "admin": 4}` em `app/auth/dependencies.py`). Assim, uma rota que exige `nerd` libera automaticamente os papéis acima dele (`require_papel_minimo`). As ações que são só de `admin` usam uma segunda regra, `require_admin`, que exige o papel exato. Os cenários viraram testes automatizados, tanto os casos que devem dar certo quanto os que devem ser barrados: `tests/test_rols.py` (`cinefilo` não comenta nem favorita; `nerd`, `stalker_do_tomhanks` e `admin` sim), `tests/test_admin.py` (usuário comum recebe 403 nas rotas de moderação; `admin` lista e apaga dados

de qualquer usuário e promove usuários) e `tests/test_quiz.py` (`cinefilo` e `nerd` não acessam o quiz; `stalker_do_tomhanks` e `admin` sim; `round_id` inválido é recusado). As regras ficaram documentadas numa tabela no README. Na implementação, também precisei corrigir campos que faltaram no commit da tela do quiz (d247b73) e o proxy de desenvolvimento do Angular para as rotas novas (a50ae97).

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	25/09/2026 13:49
Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/08359fc

O que foi feito. Um microserviço novo, o `log-service/`, grava eventos de auditoria num Redis Stream (XADD para gravar, XREVRANGE para consultar). Nem ele nem o Redis têm porta publicada. Catálogo e auth-service nunca escrevem no Redis diretamente: mandam um `POST /logs` para o `log-service` de forma fire-and-forget, com timeout curto, para que uma queda do serviço de log não quebre a ação do usuário. São auditados login, logout, favoritar, comentar, apagar comentário por moderação e toda tentativa negada por permissão (`acesso_negado: {papel}`). Essa última é registrada dentro das próprias dependências do RBAC, o que cobre todas as rotas protegidas de uma vez. Cada evento tem `usuario_id`, `acao`, `timestamp` (gerado pelo `log-service`) e `ip`.

A consulta (`GET /logs`) exige `admin`. Como o `log-service` não é público, o catálogo oferece `GET /admin/logs`: barra com 403 quem não é `admin` e repassa o token original para o `log-service`, que confere de novo. O serviço veio em ae6ba68 (12:33), os eventos e a consulta em 08359fc (13:49), a documentação e as evidências em 8d6049c (14:10) e o Redis e o `log-service` foram adicionados à stack do Portainer em bb7ca67 (14:45).

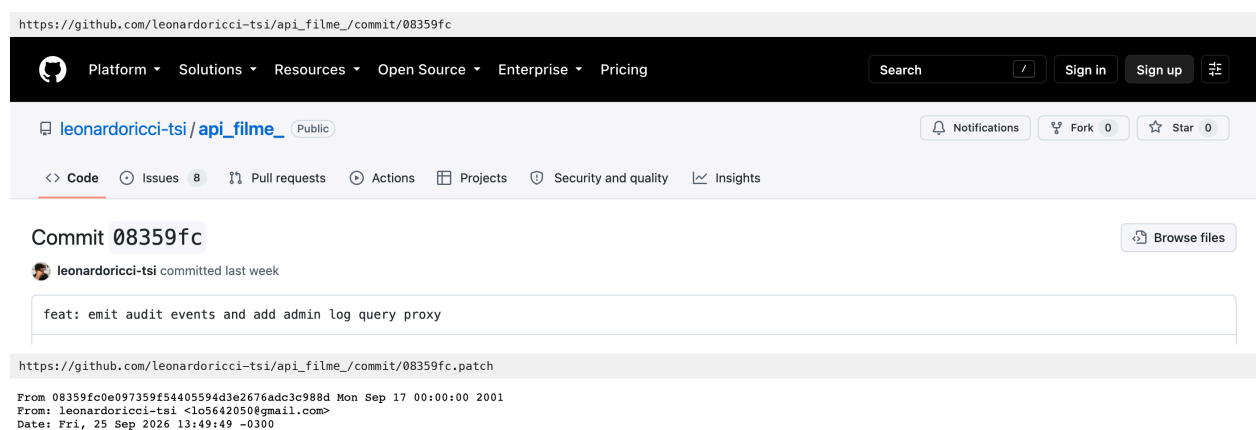


Figura 15: Atividade 5 — commit 08359fc no GitHub (repositório, hash, data e hora)

Parameters

NameDescription

limitinteger(query)

5

maximum: 1000
minimum: 1

authorizationstring | (string | null) (header)

authorization

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8000/admin/logs?limit=5' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIzMmIsInJvbmUiOiJhZG1pbSI6Im5vbWUiOiJBZG1pbSBkdWRpdG9yaWEiLCJleHAiOjE3OTAzNTkwMjR9.ky
```

Request URL

```
http://localhost:8000/admin/logs?limit=5
```

Server response

CodeDetails

200

Response body

[
 {
 "usuario_id": 33,
 "acao": "logout",
 "timestamp": "2026-09-25T16:59:36.038002+00:00",
 "ip": "184.193.200.226"
 },
 {
 "usuario_id": 33,
 "acao": "acesso_negado:admin",
 "timestamp": "2026-09-25T16:59:36.017811+00:00",
 "ip": "184.193.200.226"
 },
 {
 "usuario_id": 33,
 "acao": "comentar:13",
 "timestamp": "2026-09-25T16:59:35.991923+00:00",
 "ip": "184.193.200.226"
 },
 {
 "usuario_id": 33,
 "acao": "favoritar_filme:13",
 "timestamp": "2026-09-25T16:59:35.059233+00:00",
 "ip": "184.193.200.226"
 },
 {
 "usuario_id": 33,
 "acao": "login",
 "timestamp": "2026-09-25T16:59:33.503368+00:00",
 "ip": "172.21.0.5"
 }
]

Response headers

content-length: 545
content-type: application/json
date: Fri, 25 Sep 2026 17:00:57 GMT
server: uvicorn

Responses

CodeDescriptionLinks

200Successful ResponseNo links

Media typeapplication/json

Controls Accept header.

Example ValueSchema

"string"

422Validation ErrorNo links

Media typeapplication/json

Example ValueSchema

{

Figura 16: Atividade 5 – admin consultando o log de auditoria

Dificuldades e como foram resolvidas. Implementei os logs de auditoria com o Redis num container próprio, registrando ações sensíveis e tentativas de acesso negadas. O desafio foi fazer esse registro funcionar de forma confiável sem sobrecarregar o sistema nem comprometer o funcionamento dele. Resolvi em quatro frentes. A primeira é o envio fire-and-forget: `registrar_evento()` em `app/services/log_client.py` manda o evento ao `log-service` com timeout curto (5 s) e engole qualquer erro, então favoritar, comentar ou logar nunca falham porque o serviço de log caiu. A segunda é o custo: o cliente HTTP é criado uma vez e reaproveitado entre as requisições, e o Redis Streams grava cada evento com um `XADD`, operação feita para escrita rápida e em alto volume. A terceira é a cobertura sem duplicar código: as tentativas negadas são registradas dentro de `require_admin` e `require_papel_minimo`, então todas as rotas protegidas ficam cobertas de uma vez. A quarta é a durabilidade: o Redis grava num volume (`redis-data:/data` no `docker-compose.yml`), então os eventos sobrevivem a um reinício do container. O comportamento está coberto em `tests/test_audit_log.py`.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	29/09/2026 20:37
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/a8a6aae

O que foi feito. Cada usuário tem uma página de perfil com nome, foto, bio e filmes favoritos. A foto vai para um object storage compatível com S3 e o MySQL guarda só a chave do objeto (`perfis.foto_key`). No lugar do MinIO foi usado o Garage, troca liberada pelo professor porque a edição community do MinIO foi arquivada. O código usa `boto3` como em qualquer S3. O upload é validado pelos bytes do arquivo com `Pillow`, e não pela extensão: até 2 MB (413 acima disso), só JPEG, PNG e WEBP (415 para o resto) e limite de pixels contra decompression bomb. A imagem é reduzida e salva de novo, o que descarta metadados EXIF como a localização GPS. A exibição usa URL pré-assinada válida por 15 minutos, e o bucket nunca fica público. Só o dono edita o próprio perfil: `require_dono_do_perfil` compara o ID da URL com o sub do JWT, devolve 403 para qualquer outro, inclusive admin, e registra a tentativa no log de auditoria.

Etapas: 6ad4ca1 (Garage e cliente S3) e 575dead (perfis) em 25/09, depois be10862 (upload com validação) e b2157be (tela de perfil) em 29/09. O commit escolhido, a8a6aae, faz a foto aparecer em produção: o catálogo passa a assinar as URLs com o próprio domínio e a repassá-las ao Garage (`app/routers/storage_proxy.py`), além de colocar o Garage no Portainer e adicionar as evidências. Depois vieram a8c1a2a (correção na documentação) e bb1a8a9 (foto no avatar do cabeçalho, 30/09).

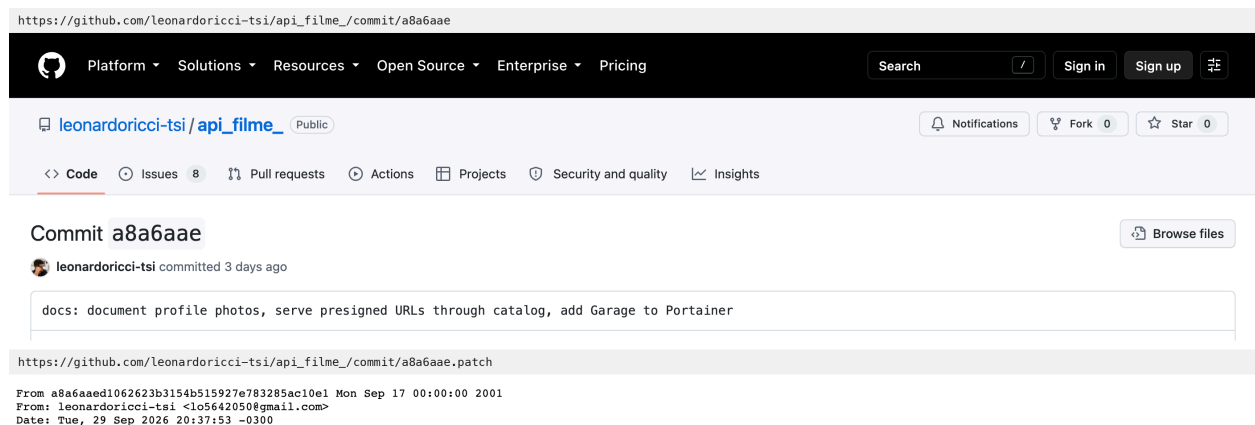


Figura 17: Atividade 6 — commit a8a6aae no GitHub (repositório, hash, data e hora)

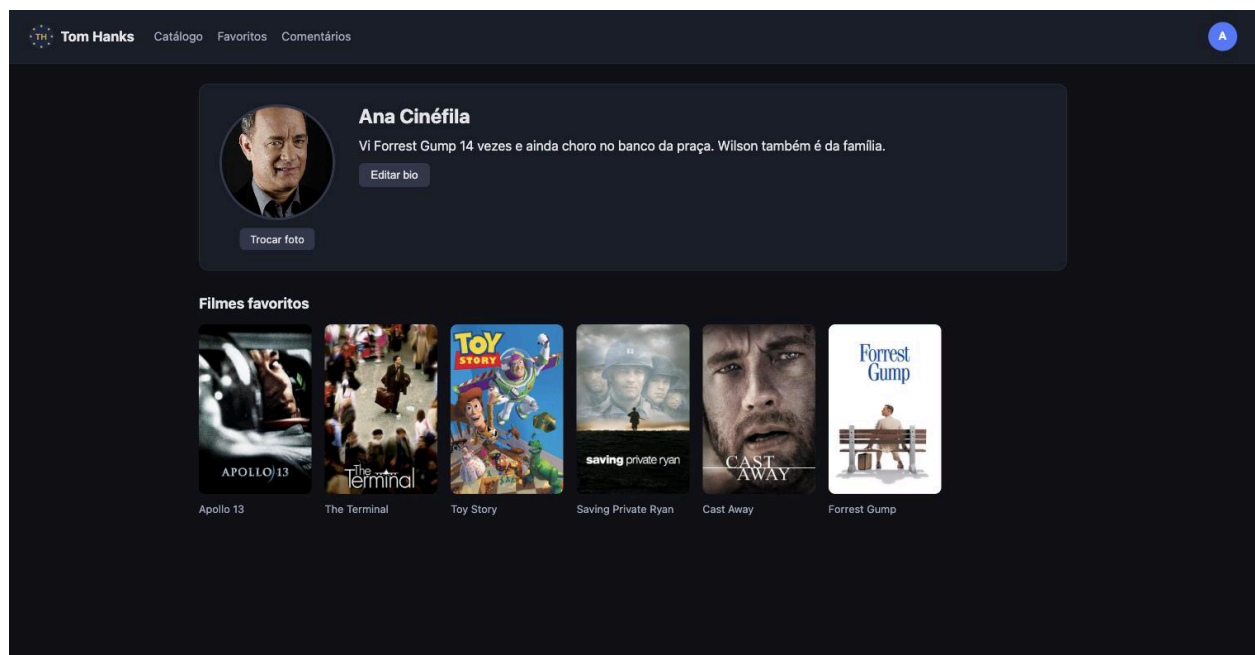


Figura 18: Atividade 6 — perfil com foto enviada ao Garage

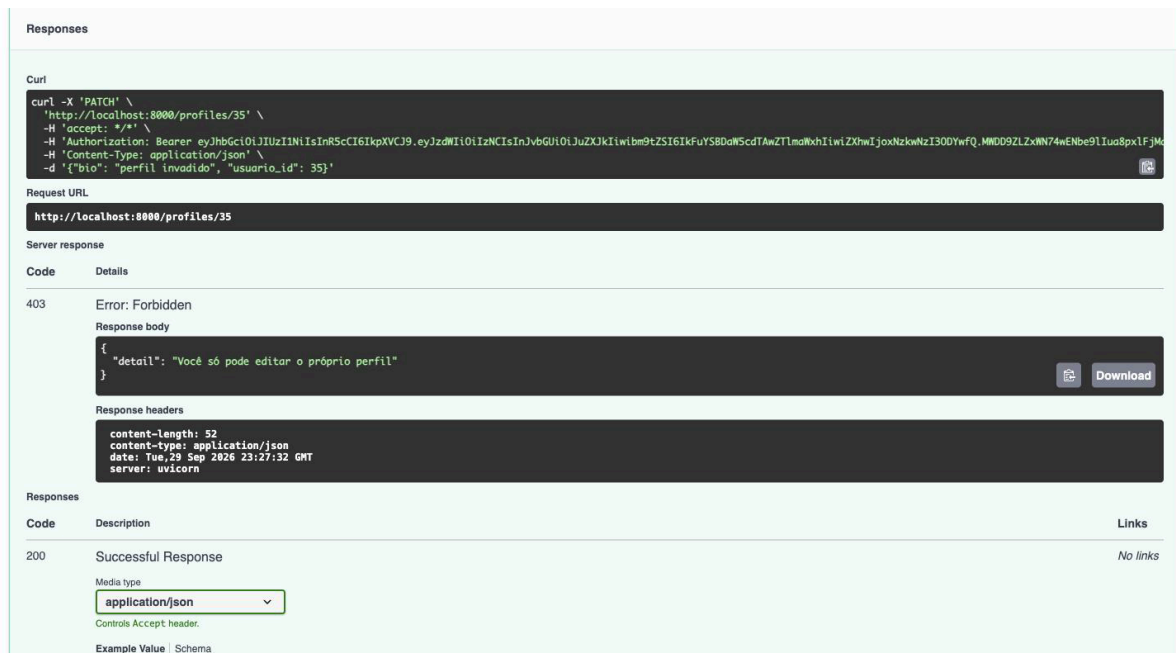


Figura 19: Atividade 6 — 403 ao tentar editar o perfil de outro usuário

Dificuldades e como foram resolvidas. Para guardar as imagens, tentei primeiro o MinIO, mas não funcionou, então migrei para o Garage num container dedicado (com autorização do professor). As tentativas com o MinIO ficaram só no meu ambiente local e não chegaram a ser commitadas. O mais complicado foi a diferença entre HTTP e HTTPS e entre portas: o servidor só libera a porta do catálogo (HTTPS), o Garage fala HTTP na rede interna e uma página HTTPS não carrega imagem de um endereço HTTP. A solução foi usar dois clientes S3 em `app/services/storage.py`. Um fala com o Garage pela rede interna (`S3_ENDPOINT_URL`). O outro só assina as URLs pré-assinadas com o domínio público do catálogo (`S3_PUBLIC_URL`, assinatura `s3v4` e endereçamento por caminho). O navegador pede a foto ao próprio catálogo, e `app/routers/storage_proxy.py` repassa o pedido ao Garage com o caminho e a query exatamente como chegaram e com o mesmo Host que entrou na assinatura. Assim, quem valida a assinatura e a expiração continua sendo o Garage, e ele segue sem porta publicada. Também descartei a opção de bucket público, porque o Garage não tem bucket policy, e corrigi a documentação quando vi que uma URL expirada devolve 400, e não 403 (a8c1a2a).

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	25/09/2026 15:04
Situação	entregue
Evidência	Repositório público no GitHub — commit com data e hora, README mencionando <code>github.com/siriani</code> (print na ficha da atividade 2), Swagger UI do catálogo e do <code>auth-service</code> , “Try it out” executado
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/e480256

O que foi feito. Os três serviços (catálogo, auth-service e log-service) são FastAPI, então o Swagger UI e a spec OpenAPI já existiam. Esta atividade documentou os erros: por padrão o FastAPI só descreve a resposta de sucesso, e cada 401, 403, 404, 409 ou 502 aparecia como “Undocumented”. Agora cada rota declara `responses=` com descrição e exemplo do erro que de fato levanta (`app/openapi_responses.py` e equivalentes nos outros serviços). O Swagger do catálogo é público em produção (`/docs`). Os do auth-service e do log-service, que não têm porta publicada, foram exportados para `docs/openapi/auth-service.json` e `docs/openapi/log-service.json`. Executando o auth-service localmente, o Swagger UI dele fica disponível em `localhost:8001/docs`.

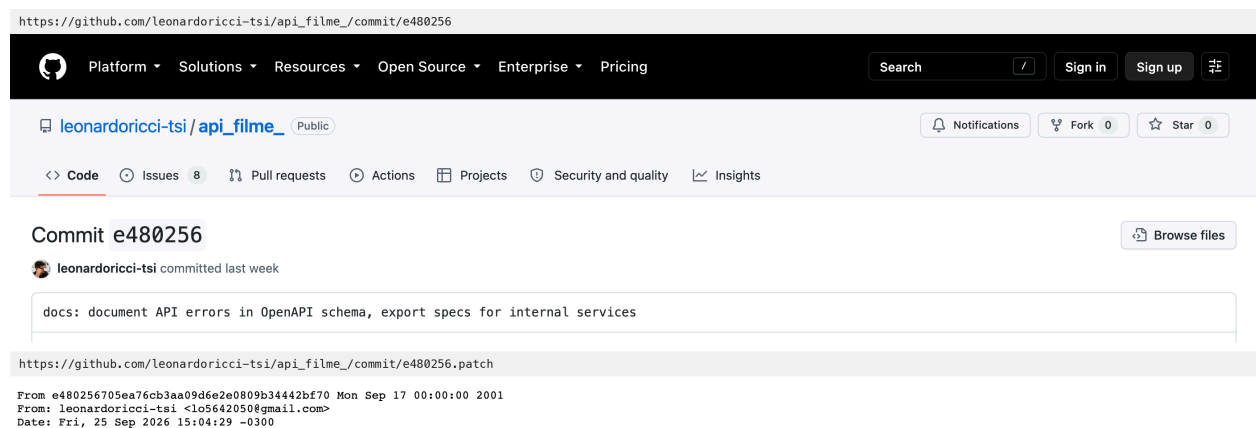


Figura 20: Extra E1 — commit e480256 no GitHub (repositório, hash, data e hora)

GET

/admin/commentsListar Todos Comentários

Só admin: lista comentários de TODOS os usuários (moderação), não só os do chamador — diferença de autorização que o role do JWT decide.

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'https://leonardo-oliveira-isw@55.lapps.studio/admin/comments' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlZMSisInjVjbGU0IjuzZXkiIlwiYm9tZSI6IHRlc3RIIEF1ZGI0b3JpYSIsImV4cCIGMTc5MDMzMjk1MX0_.Ew' 
```

Request URL

https://leonardo-oliveira-isw@55.lapps.studio/admin/comments

Server response

CodeDetails

403Error: response status is 403

Response body

```
{
  "detail": "Acesso restrito a admins"
}
```

Response headers

```
alt-svc: h3=":443"; ma=86400
cf-cache-status: DYNAMIC
cf-ray: m40bf11e3f684f8c-GRU
content-length: 37
content-type: application/json
date: Fri, 25 Sep 2026 18:02:47 GMT
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
priority: u=1,i
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=nbOUCW7KKubhqkqS4ScSSnaB1jvQWskdcHSjMTSVldmsndt7Vt2Blxvf4HYWSKf0bzky2PG/CjMLJgSXsm813zPvyMYTKLNOMAGr85vCAqugxqE4Hyo8Quk4Pnlor96yhcmyHyYHRTMpJ3oMdDoxnIXzhq56e9p9M35WMpdK9To7MJt2F"}]}
server: cloudflare
server-timing: cfExtPri
```

Responses

CodeDescriptionLinks

200Successful ResponseNo links

Media type

application/json

Controls Accept header.

Example ValueSchema

```
[{"id": 0, "tmdb_movie_id": 0, "titulo": "string", "poster_path": "string", "nome_usuario": "string", "texto": "string", "criado_em": "2026-09-25T18:02:47.429Z", "meu": true}]
```

Figura 21: Extra E1 – “Try it out” executado no Swagger público: 403 real, documentado

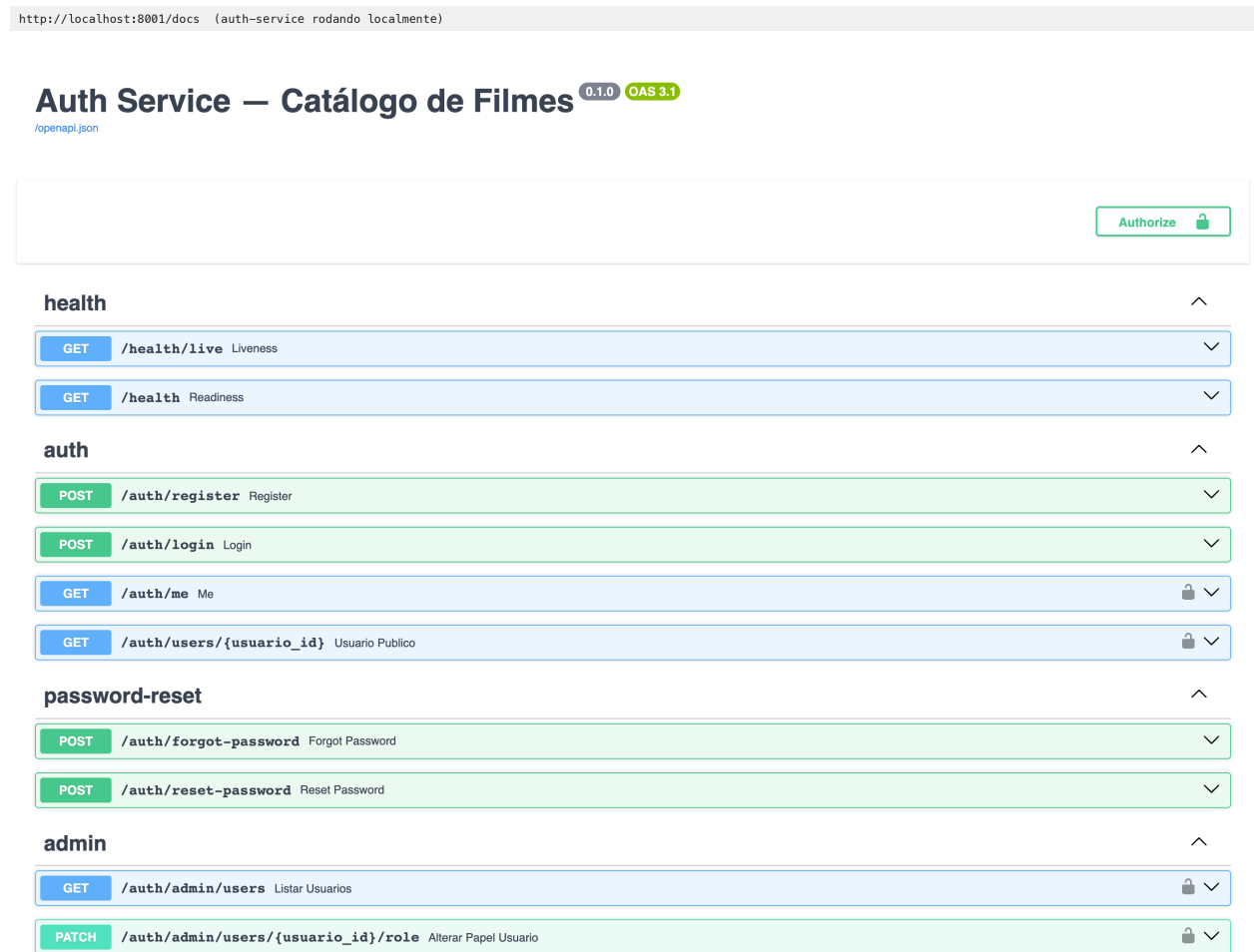


Figura 22: Extra E1 — Swagger UI do auth-service (segundo serviço documentado), executado localmente em localhost:8001/docs

Dificuldades e como foram resolvidas. Foi a atividade mais tranquila, porque eu já tinha experiência com FastAPI, que gera a documentação OpenAPI nativamente. A única dificuldade foi detalhar e organizar bem a documentação dos endpoints, principalmente os erros, que o FastAPI não documenta sozinho. Resolvi isso criando um módulo com as respostas de erro reutilizáveis (`app/openapi_responses.py`, com equivalentes no auth-service e no log-service), com descrição e exemplo de payload iguais ao `detail` que o código realmente devolve, e aplicando esses blocos em cada rota com `responses=`. Assim, o mesmo erro fica descrito do mesmo jeito em todas as rotas, sem repetir o mesmo dicionário. Os dois serviços internos não têm Swagger acessível de fora, então exportei a spec deles para `docs/openapi/`.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	30/09/2026 15:34
Situação	entregue
Evidência	Repositório público no GitHub — commit com data e hora, README mencionando <code>github.com/siriani</code> (print na ficha da atividade 2), <code>.github/workflows/ci.yml</code> , print do deploy automático

Link

https://github.com/leonardoricci-tsi/api_filme_/commit/e424517

O que foi feito. O workflow `.github/workflows/ci.yml` roda a cada push: testes do catálogo e do log-service (pytest), build das quatro imagens Docker e um smoke test que sobe a stack inteira com um MySQL descartável, roda as migrations do zero e executa `scripts/smoke-test.sh` (cadastro, login, upload de foto, 403 no perfil alheio, eventos de auditoria). Se tudo passa e o push é na main, as imagens são publicadas no GHCR com a tag do commit (sha-xxxxxxx), e o job de deploy faz um commit do robô (deploy: sha-... [skip ci]) atualizando o `docker-compose.portainer.yml`. A stack no Portainer é ligada ao repositório por GitOps e coloca a nova versão no ar sozinha. Etapas: 63dd054 (testes e build), 2a42d76 (smoke test), 8c6671e (GHCR) e e424517 (deploy GitOps). Documentação e evidência em 7b185e1 e fdfed9f.

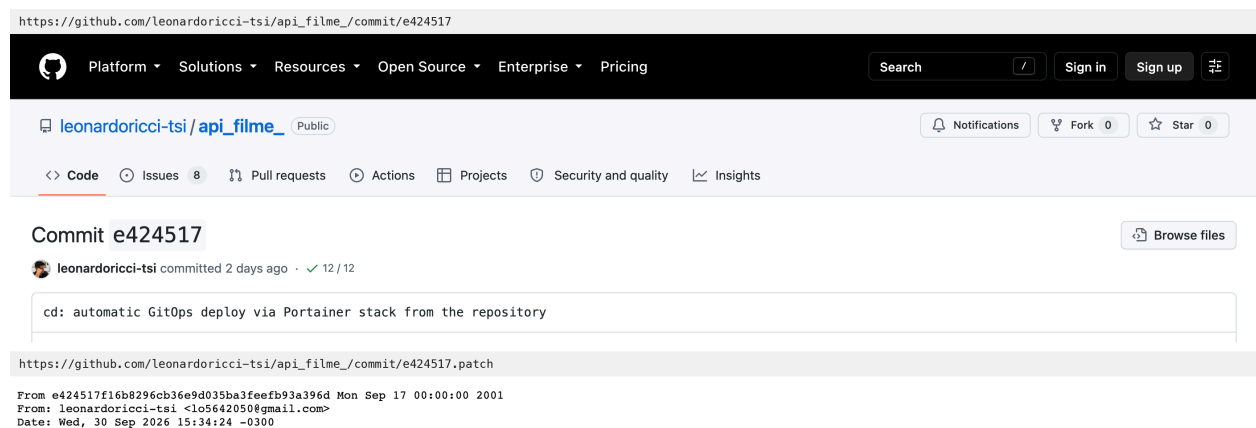


Figura 23: Extra E2 — commit e424517 no GitHub (repositório, hash, data e hora)

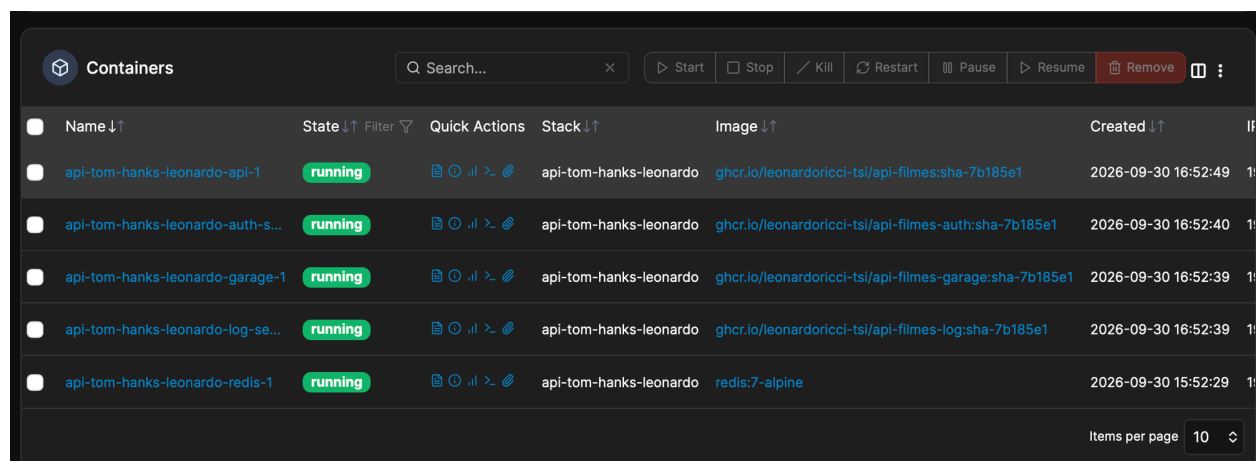


Figura 24: Extra E2 — deploy automático executado pelo pipeline

Dificuldades e como foram resolvidas. Foi a atividade em que tive mais dificuldade: comandos, scripts de deploy automático e GitHub Actions eram ferramentas novas para mim, e precisei aprender tudo do zero para montar o pipeline. Resolvi construindo o workflow em etapas, uma por commit (63dd054, 2a42d76, 8c6671e e e424517), cada uma dependendo da anterior por needs: em `.github/workflows/ci.yml`. A ordem é: testes do catálogo e do log-service, depois o build das imagens, o smoke test, a publicação e o deploy. O smoke test sobe a stack inteira com `docker-compose.ci.yml` e um MySQL descartável e roda `scripts/smoke-test.sh` contra ela. As imagens são construídas para linux/amd64, a arquitetura do servidor, diferente do meu computador, e publicadas no GHCR usando o GITHUB_TOKEN do próprio Actions, sem segredo extra. O deploy usa sed para trocar a tag das imagens no `docker-compose.portainer.yml` e faz um commit do robô com [skip ci], que o Portainer, ligado ao repositório

por GitOps, aplica sozinho. As migrations continuam manuais em produção, para não dar ao GitHub acesso ao banco.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	30/09/2026 18:29
Situação	entregue
Evidência	Repositório público no GitHub — commit com data e hora, README mencionando github.com/siriani (print na ficha da atividade 2), print do container unhealthy com o Redis fora e do painel do Grafana
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/24b40b2

O que foi feito. Cada serviço tem GET /health/live (liveness: o processo responde) e GET /health (readiness: testa as dependências e devolve 503 se uma crítica cair). No log-service, a dependência crítica é o Redis: com ele fora, o /health devolve 503 e o HEALTHCHECK do Docker marca o container como unhealthy. Quando o Redis volta, o container volta a healthy. No catálogo, só o MySQL é crítico; Garage, auth-service e log-service deixam o status degraded sem derrubar o serviço. Os três serviços expõem métricas Prometheus (prometheus-fastapi-instrumentator) numa porta interna, 9100, não publicada. No compose local, um Prometheus e um Grafana com painel provisionado pelo repositório mostram requisições por minuto, taxa de erro e latência p95. Etapas: b37a746 (health checks), 72ca482 (HEALTHCHECK no Docker), dc4cf20 (métricas) e 24b40b2 (Prometheus e Grafana). Documentação em 5c20d35.

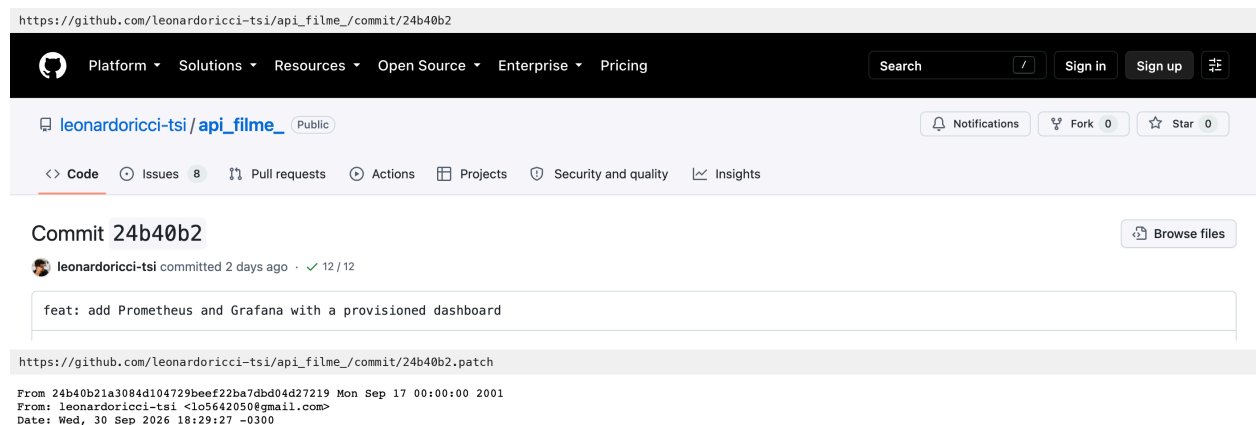
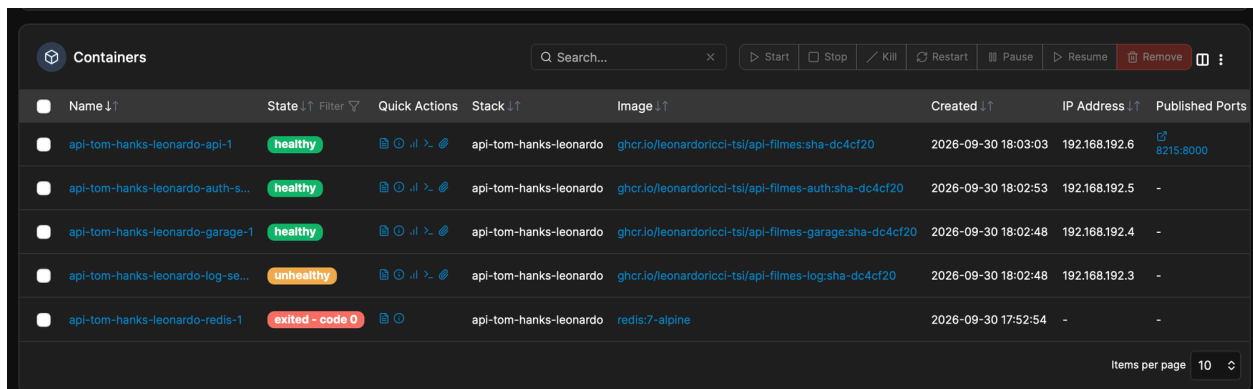


Figura 25: Extra E3 — commit 24b40b2 no GitHub (repositório, hash, data e hora)



Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports
api-tom-hanks-leonardo-api-1	healthy	[Icons]	api-tom-hanks-leonardo	ghcr.io/leonardoricci-tsl/api-filmes:sha-dc4cf20	2026-09-30 18:03:03	192.168.192.6	8215:8000
api-tom-hanks-leonardo-auth-s...	healthy	[Icons]	api-tom-hanks-leonardo	ghcr.io/leonardoricci-tsl/api-filmes-auth:sha-dc4cf20	2026-09-30 18:02:53	192.168.192.5	-
api-tom-hanks-leonardo-garage-1	healthy	[Icons]	api-tom-hanks-leonardo	ghcr.io/leonardoricci-tsl/api-filmes-garage:sha-dc4cf20	2026-09-30 18:02:48	192.168.192.4	-
api-tom-hanks-leonardo-log-se...	unhealthy	[Icons]	api-tom-hanks-leonardo	ghcr.io/leonardoricci-tsl/api-filmes-log:sha-dc4cf20	2026-09-30 18:02:48	192.168.192.3	-
api-tom-hanks-leonardo-redis-1	exited - code 0	[Icons]	api-tom-hanks-leonardo	redis:7-alpine	2026-09-30 17:52:54	-	-

Figura 26: Extra E3 — com o Redis parado, o log-service aparece como unhealthy no Portainer (produção)

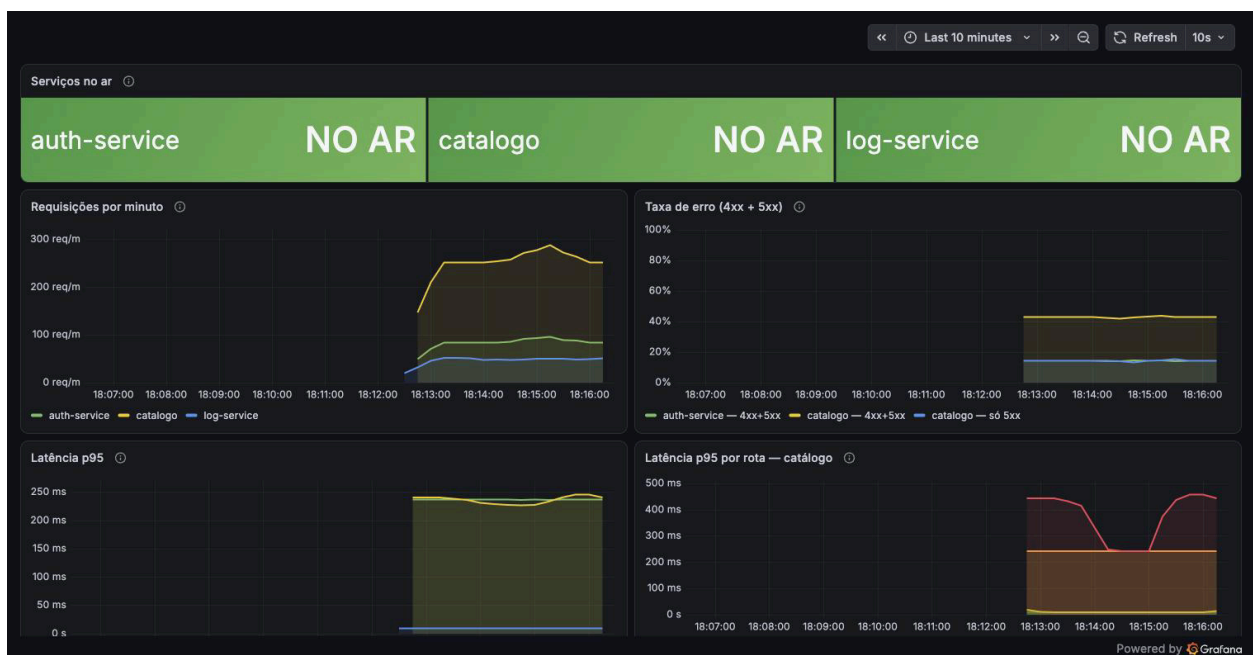


Figura 27: Extra E3 — painel do Grafana provisionado pelo repositório

Dificuldades e como foram resolvidas. Health checks e métricas com Grafana eram funcionalidades que eu nunca tinha implementado. O desafio foi pesquisar como fazer e qual a melhor forma de integrar isso a um sistema que já estava quase completo. Escolhi abordagens que não exigiam mexer nas rotas existentes. Os health checks ficaram em routers novos (`app/routers/health.py` e equivalentes nos outros serviços), que testam cada dependência e separam o que é crítico (MySQL no catálogo e no auth-service, Redis no log-service) do que não é. As métricas vieram do `prometheus-fastapi-instrumentator`, que se acopla à aplicação inteira em `app/main.py` sem alterar cada endpoint. Nele eu excluí o `/health` da contagem e ajustei as faixas do histograma para 10 ms a 5 s, porque as faixas padrão davam um p95 inútil com a latência do MySQL remoto. O `/metrics` fica numa porta interna separada (`start_http_server`, porta 9100) para não ficar exposto na internet. O HEALTHCHECK de cada container usa Python (`urllib`), porque as imagens slim não têm curl. O Prometheus e o Grafana são provisionados por arquivos do repositório (`observabilidade/prometheus/prometheus.yml` e `observabilidade/grafana/provisioning/`), sem configuração manual pela interface.

6 Melhorias de UX/UI

Além das funcionalidades pedidas em cada atividade, fui melhorando a interface do catálogo (Angular) ao longo do bimestre. A primeira versão do frontend era funcional, mas genérica: telas escuras com um formulário centralizado, sem identidade visual, e com o catálogo como única tela depois do login. As mudanças se concentraram em três commits de 29/08/2026 (03daeea, 90d6ea7 e 6690ec0) e foram complementadas depois pelo menu do usuário com papel e foto (687aab5, b2157be e bb1a8a9).

Para as comparações abaixo, executei localmente o código do commit 687aab5 (27/08/2026, último antes da reformulação visual), com o backend daquela época e o mesmo banco de dados. Os prints “depois” são da versão atual em produção. Os dois lados usam a mesma conta de teste e os mesmos dados ao vivo da TMDb, então a diferença que aparece é só a da interface.

6.1 Página inicial pública

Antes, quem abria o site caía direto na tela de login, sem saber do que se tratava o sistema. Em 03daeea criei uma landing page pública (features/landing/), com uma chamada principal, botões “Criar conta grátis” e “Já tenho conta”, cards explicando o que o catálogo oferece (catálogo completo, favoritos e comentários) e a seção “Escolha o seu nível de fã”, que apresenta os papéis do sistema como planos. Os guards de rota foram ajustados para que o visitante veja essa página antes do login.

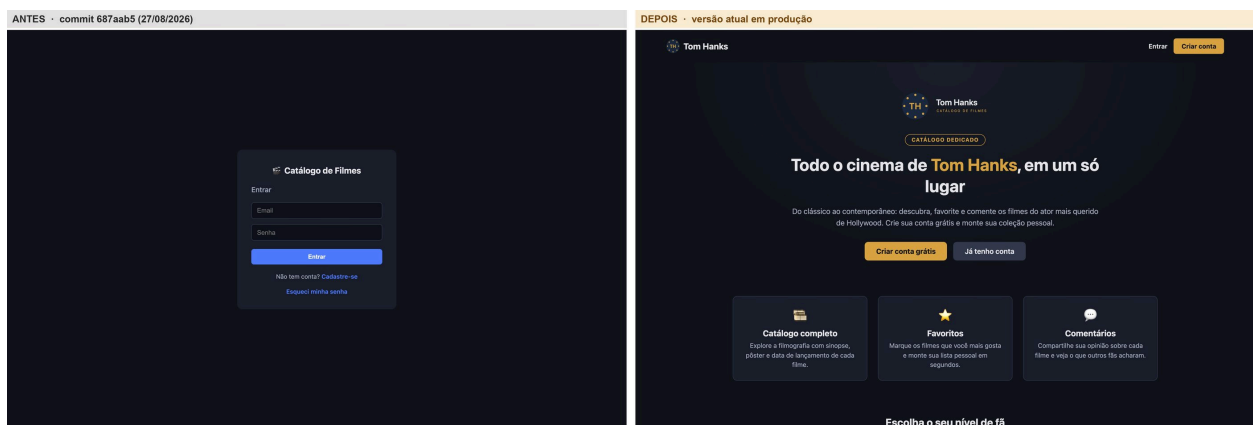


Figura 28: UX/UI — entrada do site: antes ia direto para o login; depois, landing page pública

6.2 Identidade visual e telas de login e cadastro

Em 90d6ea7 criei a identidade visual do projeto: logotipo próprio em SVG (logo.svg, logo-icon.svg, logo-claquete.svg e logo-faixa.svg em frontend/public/), usado no cabeçalho, nas telas de autenticação e como ícone da aba. As telas de login, cadastro, esqueci minha senha e redefinir senha ganharam um fundo ilustrado de sala de cinema (fundo-login.svg), campos com ícone, botão principal na cor de destaque (dourado) e o link “Esqueci minha senha” junto do campo de senha, onde o usuário procura por ele. Os estilos foram centralizados em auth-shared.css, para que as quatro telas fiquem consistentes.

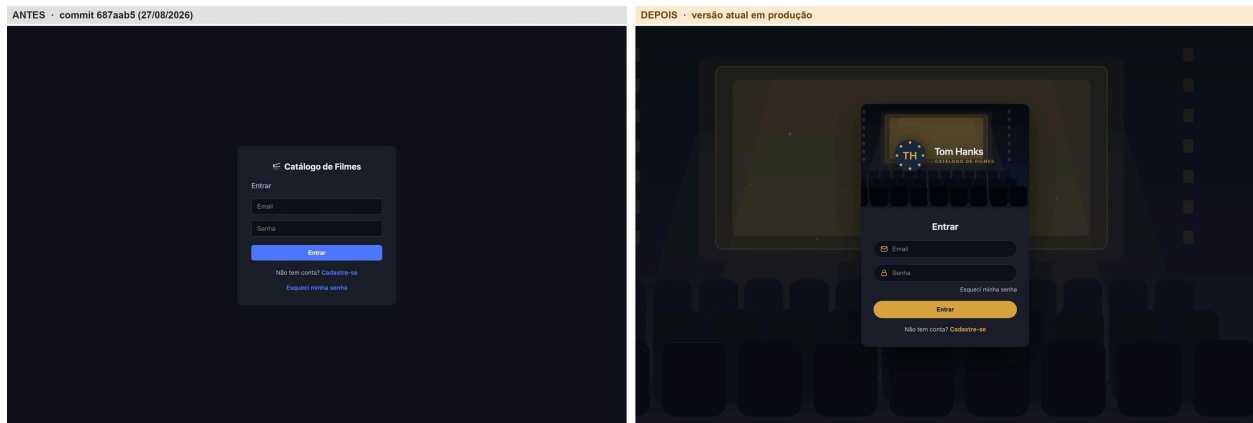


Figura 29: UX/UI — tela de login antes e depois da identidade visual

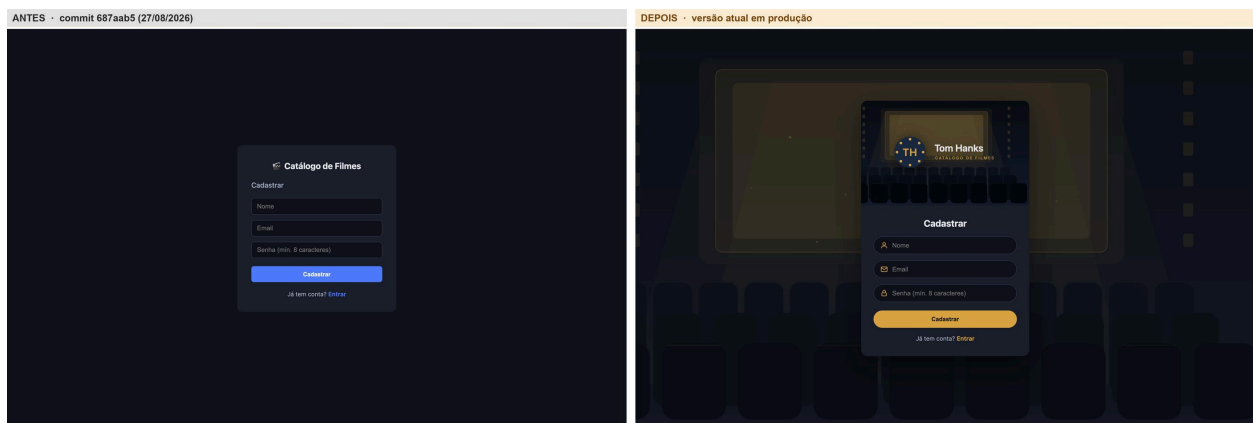


Figura 30: UX/UI — tela de cadastro antes e depois

6.3 Catálogo: busca, paginação e filmes sem pôster

Em 6690ec0 o catálogo ganhou um campo de busca por título e paginação (botões “Anterior” e “Próxima” e números de página), que antes não existiam: todos os filmes eram carregados de uma vez. Os filmes sem pôster, que antes apareciam misturados como um card vazio (“Sem pôster”), passaram a ir para o fim da lista. Os cards ficaram mais limpos: o botão “Comentários” saiu de cada card, porque o card inteiro passou a ser clicável, e ficou só o botão de favoritar, que muda de cor quando o filme já é favorito.

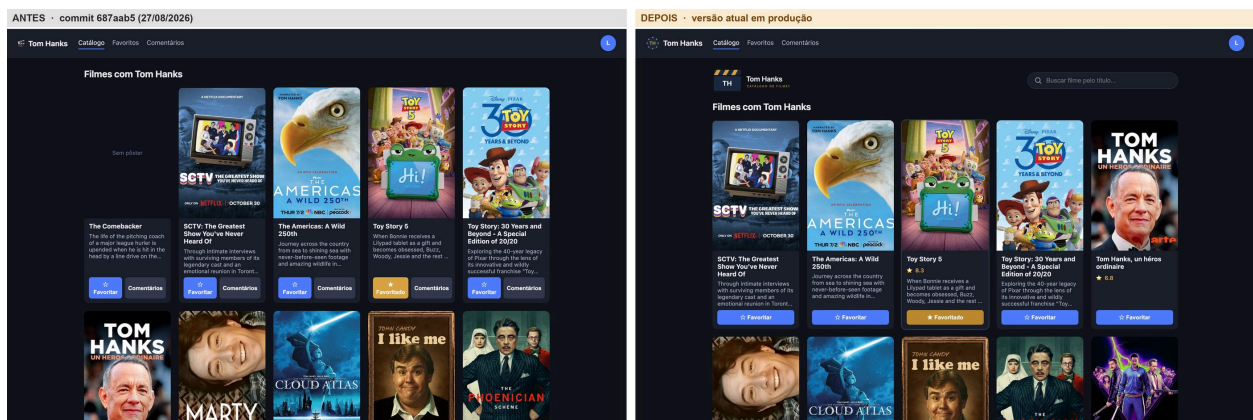


Figura 31: UX/UI — catálogo antes e depois: busca, cabeçalho com a marca e cards simplificados

6.4 Detalhes do filme

Antes, o botão “Comentários” abria um diálogo simples, só com os comentários e um campo de texto. Também no commit 6690ec0, clicar no card passou a abrir um modal de detalhes com pôster, nota, sinopse completa e três abas: Comentários (agora com o nome de quem comentou), Onde assistir e Elenco. O modal pode ser fechado pelo botão de fechar (X) ou clicando fora dele, e o card também abre com a tecla Enter, o que ajuda quem navega pelo teclado.

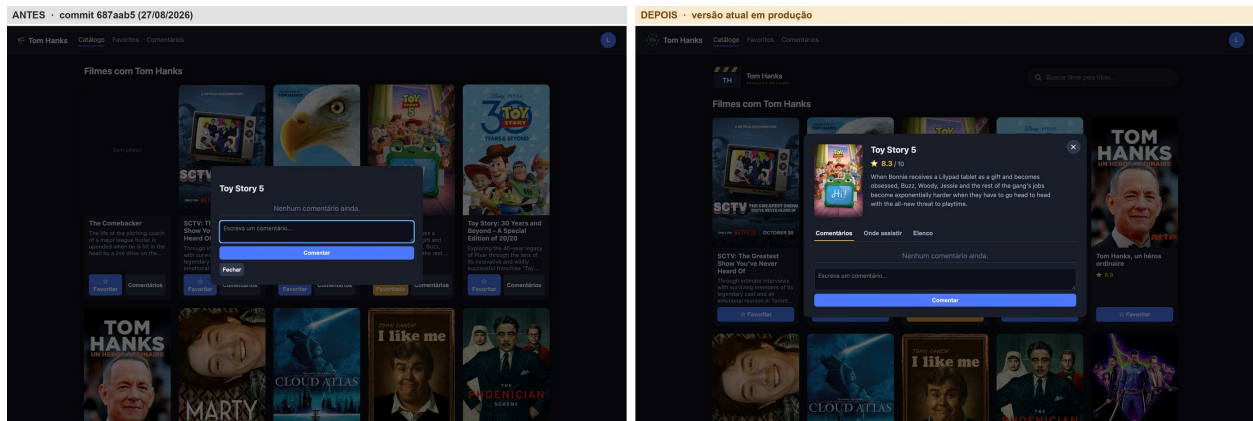


Figura 32: UX/UI — detalhes do filme: antes só comentários; depois, modal com sinopse, nota e abas

6.5 Menu do usuário

O menu aberto pelo avatar no cabeçalho passou a mostrar o papel do usuário como uma etiqueta (687aab5, adaptada aos novos papéis da atividade 4) e ganhou o atalho “Meu perfil”, que leva à página de perfil criada na atividade 6 (b2157be). Em bb1a8a9, o avatar passou a exibir a foto de perfil do usuário quando ele tem uma, em vez de só a inicial do nome.

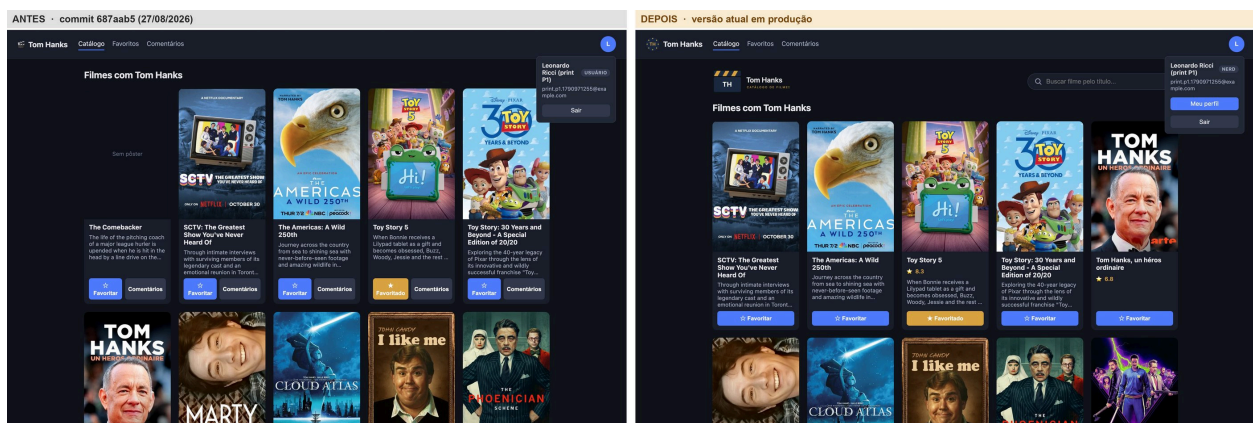


Figura 33: UX/UI — menu do usuário: depois, com o papel atualizado e o atalho para o perfil

7 Considerações finais

O bimestre mostrou na prática a diferença entre um programa que funciona na própria máquina e um sistema publicado. O mesmo catálogo passou por separação em serviços, controle de acesso no servidor, auditoria, object storage e automação de deploy, e cada passo exigiu uma decisão de arquitetura explícita: o que fica público, quem é dono de cada dado, o que é crítico e o que pode falhar sem derrubar o resto.

A parte mais difícil foi lidar com as restrições do ambiente real: um único banco por aluno, um único domínio HTTPS no servidor e a diferença de arquitetura entre o computador de desenvolvimento e o

servidor. Para o próximo bimestre, fica o hábito de documentar as decisões junto com o código e de automatizar a verificação desde o início.

Durante o bimestre, tivemos contato com diversas tecnologias, como Docker, FastAPI, Redis, object storage compatível com S3, GitHub Actions, Prometheus e Grafana, e pudemos entender o funcionamento de cada ferramenta. Nesse processo, as ferramentas de IA generativa ajudaram no desenvolvimento: deixaram mais rápido o estudo de bibliotecas que eu não conhecia, a pesquisa de soluções e a revisão do código. Assim, conseguimos ampliar o leque de oportunidades e de conhecimentos para os próximos desafios.

O diagrama abaixo mostra como a aplicação está hoje, ao fim do bimestre. O navegador só fala com o catálogo, por HTTPS, e todos os outros serviços ficam na rede interna do Docker, sem porta publicada. O MySQL é a instância remota da disciplina, fora da stack, e o Prometheus e o Grafana rodam só no ambiente local. Embaixo está o caminho de cada mudança até a produção: o push dispara o GitHub Actions, as imagens vão para o GHCR e o Portainer aplica a versão nova.

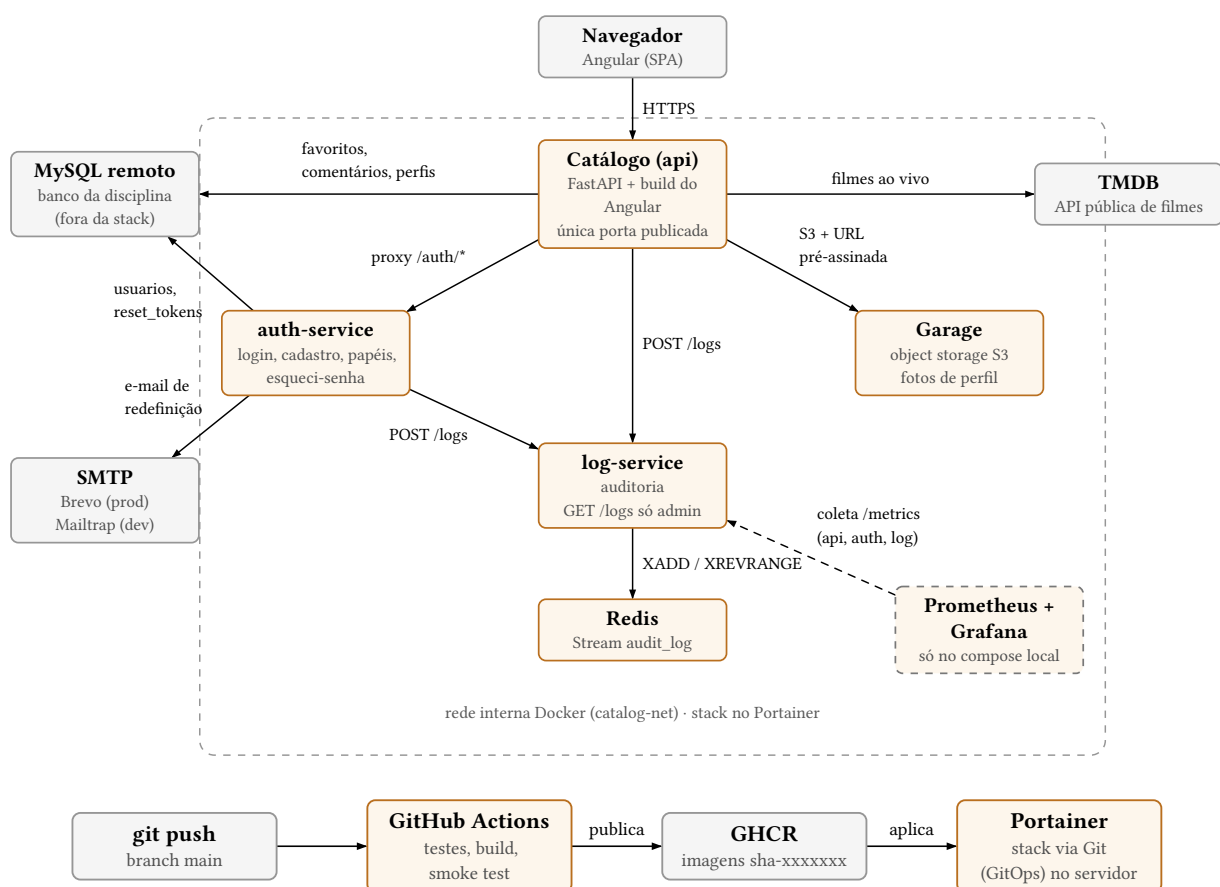


Figura 34: Arquitetura atual da aplicação (acima) e fluxo de deploy (abaixo)

8 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Documento assinado digitalmente
 LEONARDO RICCI SANTOS OLIVEIRA
Data: 06/10/2026 20:31:33-0300
Verifique em <https://validar.iti.gov.br>

Leonardo Ricci Santos Oliveira

Pompeia, 06/10/2026